

IQRASEC ACADEMY

WEB APPLICATION PENETRATION TESTING (WAPT/VAPT) INTERNSHIP

Task 4: Common-Web-Vulnerabilities

Phase 1: Practical Web Security

Sudip Kumar Chaurasiya

Internship: IqraSec Academy - WAPT/VAPT Internship

Date: 15 October 2026

Task 4: Common-Web-Vulnerabilities

2. Concept / Understanding

SQL Injection

SQL Injection occurs when user input is improperly included in a database query. This can allow an attacker to manipulate the query.

Cross-Site Scripting (XSS)

XSS occurs when an application does not properly handle untrusted input and attacker-controlled script content can be executed in a user's browser.

Authentication Vulnerabilities

Authentication vulnerabilities involve weaknesses in the way an application verifies users or manages authentication.

IDOR

Insecure Direct Object Reference occurs when an application uses user-controlled references to access objects without properly checking whether the user is authorized to access them.

Broken Access Control

Broken Access Control occurs when an application does not properly enforce what authenticated or unauthenticated users are allowed to access or perform.

CSRF

Cross-Site Request Forgery can cause an authenticated user's browser to send an unwanted request to an application.

Path Traversal

Path Traversal occurs when an application improperly handles file paths and may allow access to files outside the intended directory.

File Upload Vulnerabilities

File upload vulnerabilities can occur when an application does not properly validate or safely handle uploaded files.

Command Injection

Command Injection occurs when user input is improperly passed to operating system commands and may result in unintended command execution.

OWASP Top 10

The OWASP Top 10 provides a list of important categories of web application security risks. Studying these categories helps security testers understand common weaknesses in web applications.

3. Practical Work

Part A: TryHackMe – IDOR / Access Control

I completed the assigned TryHackMe room related to IDOR and access control. During the room, I learned how access control problems can occur when an application does not properly verify whether a user is authorized to access a particular resource. I practiced examining application requests and understanding how object references can be used by an application.

TryHackMe

Attack Save

7. Intruder attack of http://10.49.161.191

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	210			966	
1		1337	50			76	
2	0	1337	47			76	
3	1	1337	49			76	
4	2	1337	42			76	
5	3	1337	39			76	
6	4	1337	38			76	
7	5	1337	42			76	
8	6	1337	40			76	
9	7	1337	40			76	
10	8	1337	36			76	
11	9	1337	44			76	
12	10	1337	41			76	
13	11	1337	48			76	
14	12	1337	48			76	
15	13	1337	47			76	
16	14	1337	46			76	
17	15	1337	49			76	
18	16	1337				76	

Request

```

1 GET /api/parents/view_accountinfo?user_id=10 HTTP/1.1
2 Host: 10.49.161.191
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://10.49.161.191/
8 Content-Type: application/json
9 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlbnRlLnR5b2x1IiwiaWF0IjE0E3OTYyMDYyOTZ9.0q3l1lDCq5AH6FKHd8WlUP_wfbersBg_vfcIeng8ZE

```

Response

```

1 HTTP/1.1 200 OK
2 Server: nginx/1.24.0 (Ubuntu)
3 Date: Mon, 05 Dec 2026 11:58:03 GMT
4 Content-Type: application/json
5 Content-Length: 802
6 Connection: keep-alive
7 {
8   "user_id": 10,
9   "username": "Nielsa",
10  "email": "nielsa@webmail.thm",
11  "firstname": "Nielsa".

```

Inspector

Request attributes: 2

Request query parameters: 1

Request headers: 10

Response headers: 5

Event log: Finished

I also completed the required activities in the room.

tryhackme.com/room/idor-aoc2025-zl6MywQid9/congratulations?step=room-completion-share

Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali Forums OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

IDOR - Santa's Little IDOR ✓

Share your progress and stand out in the cyber security community!

Boost your visibility and highlight your dedication to professional growth

in LinkedIn WhatsApp More options

Skip, I like to celebrate in silence

tryhackme.com/room/idor-aoc2025-zl6MywQid9

Room completed: 100%

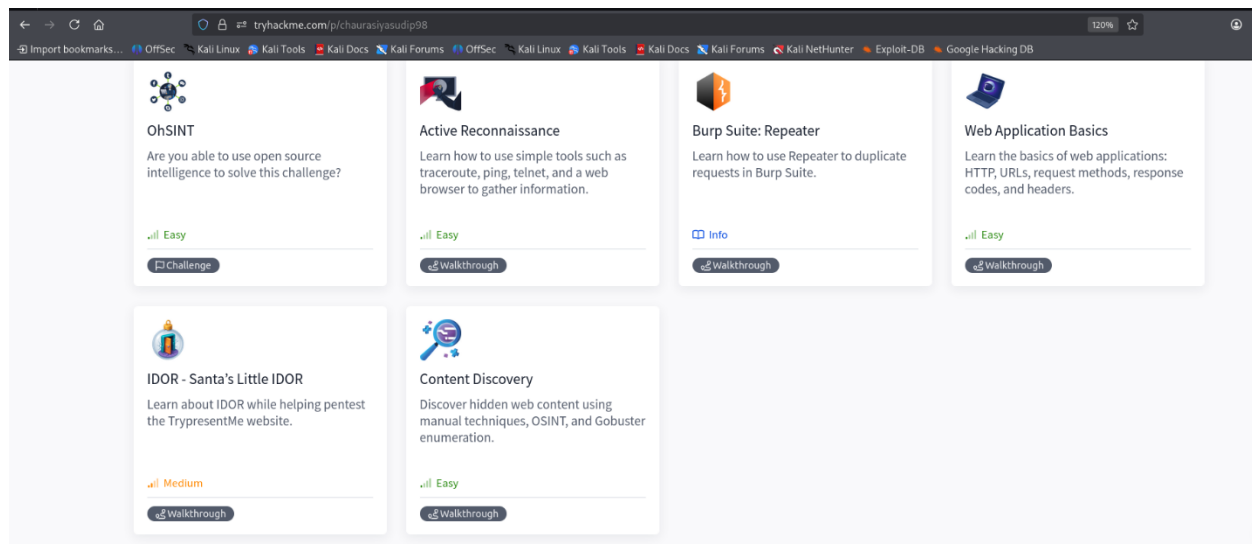
Watch on YouTube

Active machines information IP Address: 10.49.161.191

Task 1 Introduction

Task 2 IDOR on the Shelf

How likely are you to recommend this room to others?



PART B: PORTSWIGGER WEB SECURITY ACADEMY LABS

I completed two SQL Injection labs from PortSwigger Web Security Academy. Both labs were completed at the Apprentice level.

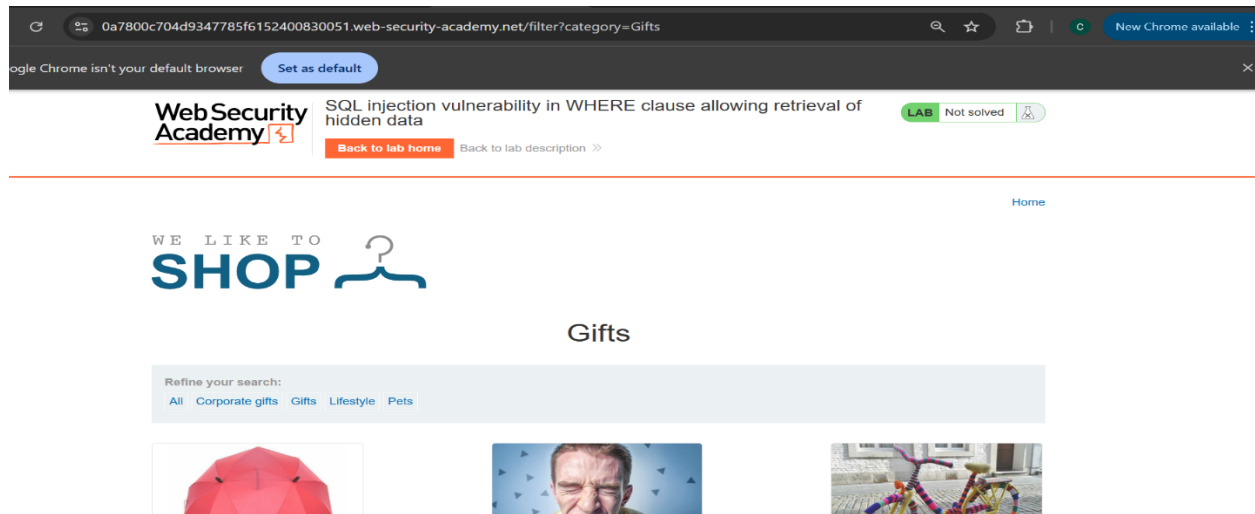
PortSwigger Lab 1: SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

Vulnerability / Concept

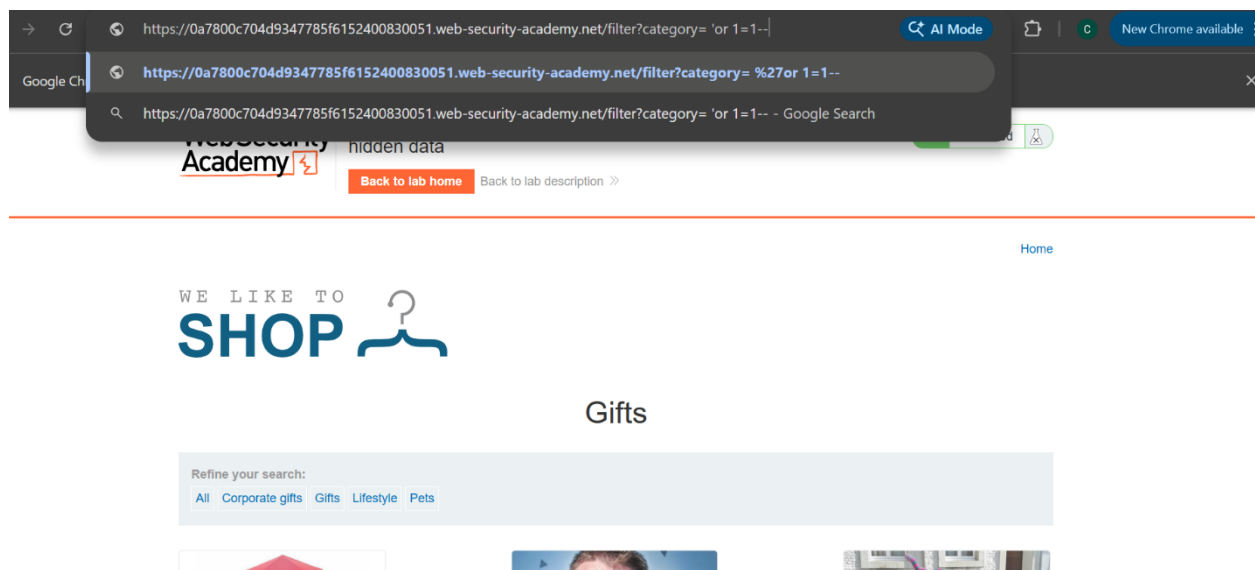
The application was vulnerable to SQL Injection because user-controlled input was included in a SQL query without being properly protected. So the vulnerability occurred in a WHERE clause used to filter products. By modifying the input it was possible to change the behavior of the SQL query and retrieve information that was intended to remain hidden.

Testing Approach

I first accessed the PortSwigger lab and examined the application's product categories and filtering functionality. I identified the parameter used to select a product category and tested how the application responded to modified input.



I then modified the request to test the SQL Injection vulnerability.



After sending the modified request then the application returned hidden products/data that were not normally displayed.



So the lab was successfully solved.

Impact

SQL Injection can allow an attacker to manipulate database queries. Depending on the application's configuration and database privileges. So this could expose unauthorized information or affect the integrity of database data.

Remediation

The application should use parameterized queries or prepared statements instead of directly including user input in SQL queries. Input validation and appropriate database permissions should also be implemented as additional security measures.

What I Learned

I learned how SQL Injection can occur in a WHERE clause when user input is not properly handled. I also learned how modifying an HTTP request can change the behavior of a vulnerable application.

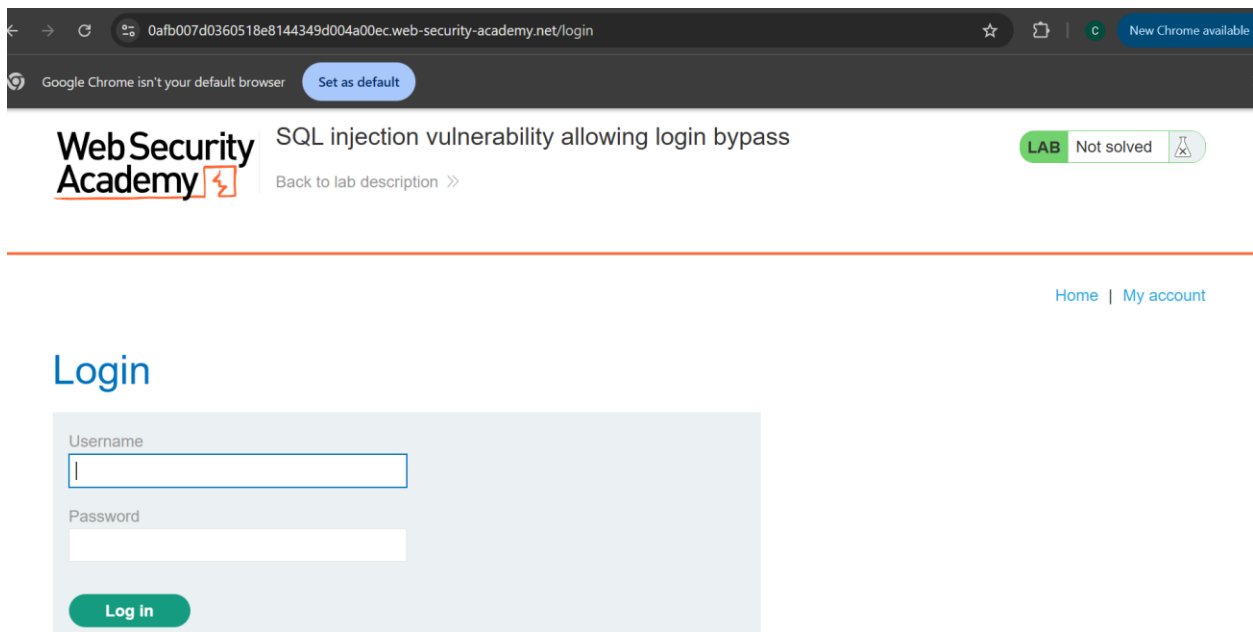
PortSwigger Lab 2: SQL Injection Vulnerability Allowing Login Bypass

Vulnerability / Concept

The login functionality was vulnerable because user input was included in a SQL query without proper protection. By manipulating the login input, it was possible to change the logic of the SQL query and bypass the normal authentication process.

Testing Approach

I first opened the login page and examined the login functionality. I entered the required information and analyzed the HTTP request generated by the application.



I then modified the request/input to test whether the login query could be manipulated.

Login

Username

administrator'--

Password

.....

Log in

After submitting the modified request, the application accepted the request and allowed the login bypass as required by the lab.

0afb007d0360518e8144349d004a00ec.web-security-academy.net/my-account?id=administrator

Google Chrome isn't your default browser [Set as default](#)

New Chrome available

Web Security Academy 

SQL injection vulnerability allowing login bypass

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab! [Share your skills!](#)   [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email

The PortSwigger lab was successfully solved.

Impact

A SQL Injection vulnerability in a login function can allow an attacker to bypass authentication and potentially gain unauthorized access to an account or application functionality.

Remediation

Developers should use parameterized queries or prepared statements for database queries. User input should never be directly concatenated into SQL statements. Authentication logic should also be designed so that user input cannot alter the intended database query.

What I Learned

I learned how a SQL Injection payload such as administrator'-- can be used in a vulnerable login form to bypass password authentication. I learned that the -- comments out the remaining part of the SQL query, which can prevent the password check from being executed. This lab helped me understand how improper handling of user input can make a login system vulnerable to SQL Injection.

4. Tools Used

The following tools and platforms were used during the practical work:

- TryHackMe
- PortSwigger Web Security Academy
- Burp Suite
- Web Browser
- Burp Proxy
- Burp Repeater
- HTTP Request and Response Analysis

These tools were used to practice web application security concepts, analyze HTTP traffic, test access control, and complete the assigned SQL Injection labs in controlled environments.

5. Evidence

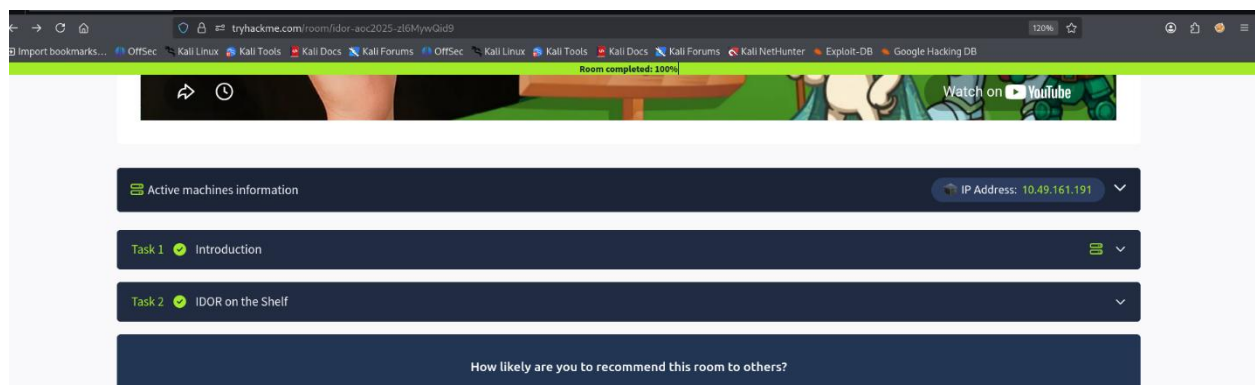
The evidence for this task includes the following practical work:

1. TryHackMe IDOR/access control practical evidence.
2. TryHackMe room completion screenshot.
3. PortSwigger SQL Injection lab screenshots.
4. Burp Suite request and response analysis.
5. Original and modified HTTP requests where applicable.
6. Server responses showing the testing results.
7. PortSwigger lab completion evidence for both completed SQL Injection labs.

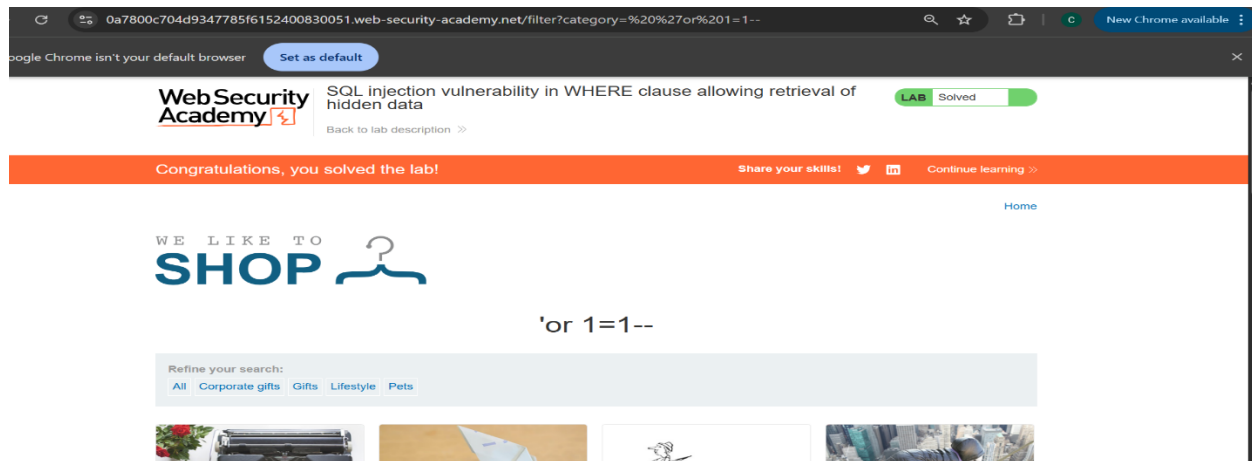
PortSwigger Labs Completed

- **SQL injection vulnerability in WHERE clause allowing retrieval of hidden data**
- **SQL injection vulnerability allowing login bypass**

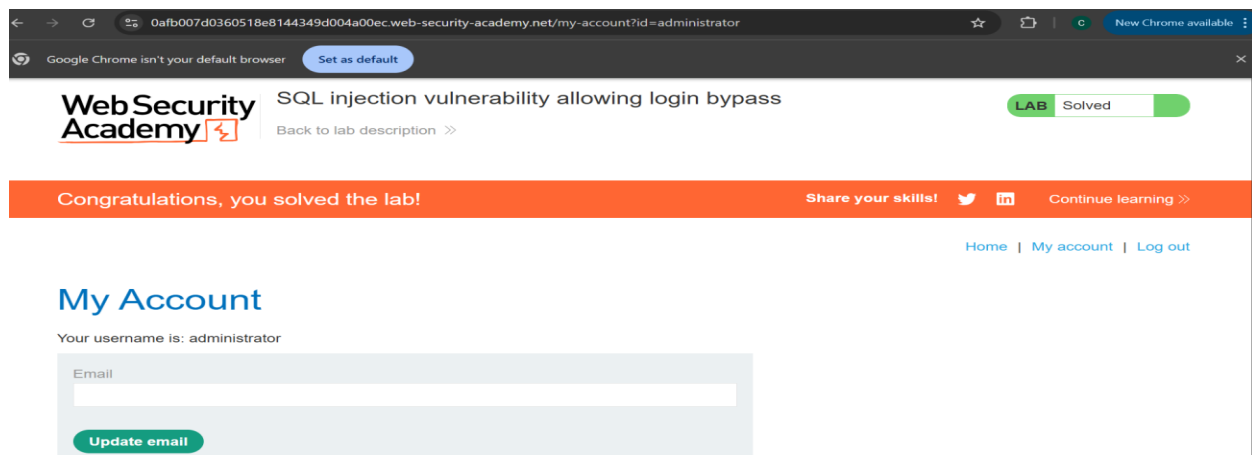
TryHackMe IDOR/access control completion evidence



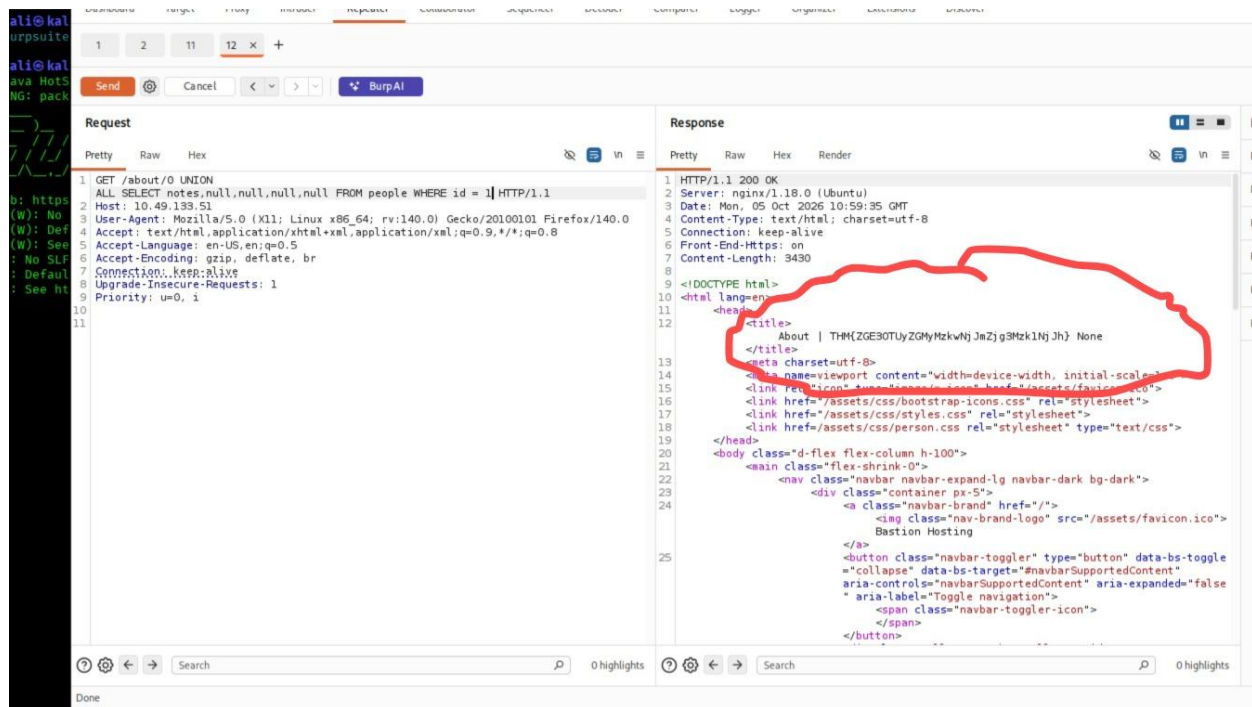
PortSwigger SQL Injection Lab 1 completion evidence



PortSwigger SQL Injection Lab 2 completion evidence



Burp Suite practical request/response evidence



6. Result

I successfully practiced common web application security vulnerabilities in controlled learning environments. The TryHackMe practical work helped me understand the importance of proper access control and how IDOR vulnerabilities can occur when an application does not correctly verify whether a user is authorized to access a resource. I also completed two SQL Injection labs on PortSwigger Web Security Academy. These labs helped me understand how improper handling of user input can affect database queries and authentication mechanisms. By using Burp Suite, I practiced analyzing HTTP requests and responses and observing how changes to requests can affect application behavior.

Overall, the practical work improved my understanding of web application security testing and the importance of secure input validation, authentication, and authorization.

7. Key Learnings

- I learned about common web application vulnerabilities.
- I learned how IDOR can occur because of missing or weak authorization checks.
- I understood the importance of proper access control.
- I learned the basic concept of SQL Injection and how unsafe input handling can affect database queries.
- I practiced SQL Injection testing through two PortSwigger Web Security Academy labs.
- I learned how SQL Injection can affect application functionality and authentication.
- I learned about XSS and how untrusted input can become a security risk.
- I learned about authentication vulnerabilities and the importance of secure authentication mechanisms.
- I understood the basic concept of CSRF and why applications need protection against unauthorized requests.
- I learned about Path Traversal and File Upload vulnerabilities.
- I learned how Command Injection can affect an application and its underlying operating system.
- I practiced using Burp Proxy and Burp Repeater for HTTP request and response analysis.
- I improved my understanding of how web applications process different requests.
- I learned that proper input validation, authentication, authorization, and secure application design are important for preventing web vulnerabilities.
- The practical labs helped me connect the theoretical concepts of web security with real-world testing techniques.